

```

#include <chrono>
#include <iostream>
using namespace std;
using clk = chrono::high_resolution_clock;
volatile int sink_int = 0; // prevent over-optimization

int main() {
    const int MAX_CAP = 100000; // fixed buffer capacity
    int n = 0; // logical size you control

    // init outside the timed block as needed...
    // for (int i=0;i<n;i++) A[i]=i;

    auto t0 = clk::now();

    // ==== WORK HERE (only the operation under test) ====

    auto t1 = clk::now();
    cout << chrono::duration_cast<chrono::microseconds>(t1-t0).count() << "\n";
}

```

Rules for every challenge

- Do all setup (filling, seeding indices, printing) outside the timed block.
- Put only the target work between t0 and t1.
- Repeat ≥ 5 trials and average. Try multiple n (e.g., 1e3, 1e4, 1e5) where it makes sense.
- Hand in: your code, a small table of times, and a 2–4 sentence explanation per part.

Challenge 1 — Insert 1 element in array

1A. Insert into empty array

- Scenario: $n = 0$, capacity = MAX_CAP. Insert at index 0.
- Predict: Any shifting? How many writes?
- Measure: Time only the insert.

1B. Insert when not full, no index given

- Scenario: $n = \text{MAX_CAP}/2$ (has spare). “Insert one element” with no index specified.
- Decide: State the fastest correct behavior you choose.
- Measure: Time that behavior.

1C. Insert at a given index (capacity exists)

- Scenario: $n = \text{MAX_CAP}/2$. Insert at:
 - beginning (0)
 - middle ($n/2$)

- end (n)
- Predict: Order fastest → slowest before running (justify with “how many elements move”).
- Measure: Time each case; report the three numbers side-by-side.

1D. Insert when full (resize)

- Scenario: $n = \text{MAX_CAP}$ (no spare).
- D1 (fixed buffer): Can you insert without overwrite? (state why/why not; no timing)
- D2 (simulate dynamic array): Allocate $2n$, copy n elements, then insert once; time only resize+copy+insert.
- Compare: Put the timing of D2 next to 1B/1C-end and explain the new cost that appears.

Challenge 2 — Remove 1 element

2A. Remove from the middle

- Scenario: $n = \text{MAX_CAP}/2$, remove at $\text{idx} = n/2$.
- Measure: Shift left to fill the gap; time only the shift.

2B. Remove from the end

- Scenario: $n = \text{MAX_CAP}/2$, remove last element.
- Measure: Perform the minimal work to drop the last element (no shifting).

2C. Remove from the beginning

- Scenario: $n = \text{MAX_CAP}/2$, remove index 0.
- Measure: Shift left; time only the shift.

2D. Edge sanity

- Scenario: $n = 1$, remove index 0.
- Measure & Note: Compare with 2A/2C briefly.

Challenge 3 — Single Question

Access by index vs Linear search — what’s fundamentally different, and how would your measurements show it?

- In one paragraph, explain whether $\text{arr}[i]$ requires scanning and how you’d prove your claim with a small timing loop (no need to provide code).
- Then contrast with linear search in best case vs worst case and what growth you expect across n .